Python

15/04/24

DATE: / /
PAGE NO.: 35

Assignment-4

22BCON1068

- Sec-A

1. Explain what the __str__ method does and why it is a useful method to include in a class.

Ans The __str__ Method in Python is a special method used to define the string representation of an object. When we are call the built-in 'str()' fun or we the 'print()' fun with an object as an argument, Python internally calls the '__str__' method of that object to obtain a string representation.

2. List the applications of special class attributes.

Ans Special class attributes in Python are attributes that have double underscores at the beginning and end of their names such as, '__init__', '__doc__', etc.

Application:

- '__init__': used as a constructor method for initializing objects of a class with initial values.
- '__doc__': Contains the documentation string or docstring for the class, providing information about the class and its methods.
- '__dict__': Contains the namespaces of the class or instances as a dictionary, allowing access to its attributes.
- '__str__': Define the "informal" or readable string representation of an object, typically used for printing or displaying to user.
- '__del__': Defines behavior when an object is about to be destroyed or deleted.
- '__call__": Enables instances of a class to be called as functions, allowing to class to act like a function.
- '__len__": Defines the behavior for obtaining the length of an object, typically used for collections like lists, tuples.
- "__repr__': Defines the "official" string representation of an object, meant to be unambiguous and suitable for debugging.

3. What is regular expression and its types?

Ans A regular expression (regex) is a sequences of characters that defines a search pattern. It's used for pattern matching within strings.

There are several types:

(i) Basic Regular Exp. (BRE)
(ii) Extended Regular Exp (ERE)
(iii) Perl-Compatible Regular Exp. (PCRE)
(iv) POSIX Reg. Exp.
(v) JavaScript Reg. Exp.
(vi) Unicode Reg. Exp.

4. Explain dir() fun. with an example.

Ans The 'dir()' function in Python returns a list of valid attributes for a specified object. It's commonly used for introspection, allowing to explore the properties and methods available for an object.

ex.

if an object of a method called is named as --dir--(), the fun must return a list of attributes that are associated with that fun.

Code:

```
lang = ("C", "C++", "Java", "Python")
att = dir(lang)
print(att)
```

Output: ['__add__', '__class__', '__class_getitem__', '__contains__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getitem__', '__gt__', '__hash__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'count', 'index']

5. What can be done with overriding a method?

Ans Method overriding allows subclasses to provide their own implementation for a method defined in the superclass. it enables customization, extension, and modification of behavior facilitating polymorphism and maintaing consistency across classes.

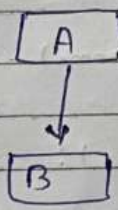
- Sec-B (3x7=21)

1. What is meant by inheritance? explain the different types of inheritances.

Ans Inheritance is defined as the mechanism of inheriting the properties of the base class to the child class.

There are five types of inheritance:

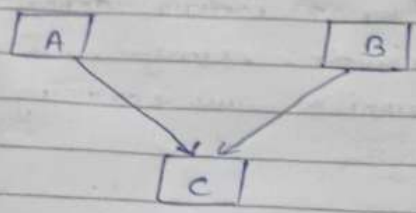
(i) Single Inheritance: in single level inheritance, a subclass inherits from only one superclass.



code:

```
class Animal:
    def sound(self):
        print("sound")

class Dog(Animal):
    def bark(self):
        print("woof")

d = Dog()
d.sound()    # output: Sound
d.bark()     # output: Woof
```

(ii) Multiple Inheritance: Multiple inheritance allows a subclass to inherit from more than one superclass.

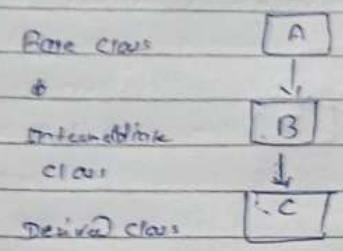


Code:

```
Class Mother:
    mothername = ""
    def mother(self):
        print(self.mothername)

Class Father:
    fathername = ""
    def father(self):
        print(self.fathername)

Class Son(Mother, Father):
    def parents(self):
        print("Father:", self.fathername)
        print("Mother:", self.mothername)

S1 = Son()
S1.fathername = "RAM"
S1.mothername = "SITA"
S1.parents()
```

Output: Father: RAM
Mother: SITA

(iii) Multilevel Inheritance : In multilevel inheritance, a subclass inherits from a superclass, and then another subclass inherits from that subclass, forming a chain.

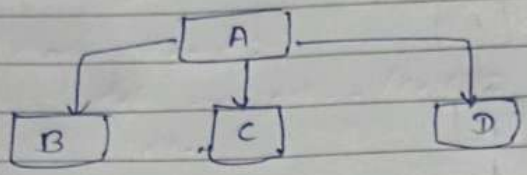


code :

```
class Father :
    surname = "Singh"
class Son(Father):
    def show(self):
        print("Akash", self.surname)
class Gson(Son):
    def Disp(self):
        print("Ankit", self.surname)

S = Son()
S.show()
Gs = Gson()
Gs.Disp()
```

Output: Akash Singh
Ankit Singh

(iv) Hierarchical Inheritance: inheritance which contain only one parent class and multiple child classes but each child classes can access parent class properly.

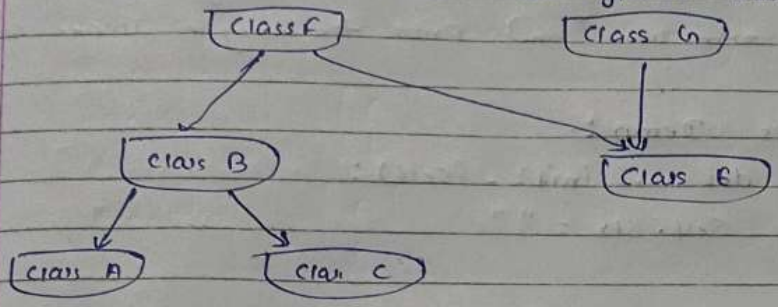


code:

```
class Parent:
    def fun1(self):
        print("This function is in parent class.")

class Child1(Parent):
    def fun2(self):
        print("This function is in child 1.")

class Child2(Parent):
    def fun3(self):
        print("This function is in child 2.")

Object1 = Child1()
Object2 = Child2()
Object1.fun1()
Object1.fun2()
Object2.fun1()
Object2.fun3()
```

op: This fun. is in Parent class.
This fun. is in Child 1.
This fun. is in Parent class.
This fun. is in Child 2.

(v). Hybrid inheritance : Hybrid inheritance is a combination of multiple inheritance and other types of inheritance.



code :

```
class School :
    def fun1(self):
        print("This is school.")
class Student1(School):
    def fun2(self):
        print("This is student 1.")

class Student2(School):
    def fun3(self):
        print("This is student 2.")

class Student3(Student1, School):
    def fun4(self):
        print("This is student 3.")

Object = Student3()
Object.fun1()
Object.fun2()
```

O/P : This is school.
This is student 1.

Q 2. Write a program to create a class named Demo. Define two methods Get_String() and Print_String(). Accept the string from user and print the string in upper case.

Ans

```
class Demo:
    def __init__(self):
        self.str = ""

    def Get_String(self):
        self.str = input("Enter a string:")

    def Print_String(self):
        print("String in uppercase:", self.str.upper())

demo = Demo()          # create a obj.
demo.Get_String()      # call the Get_String()

demo.Print_String()    # call the Print_String()
```

Output: Enter a String : ankit
String in uppercase : ANKIT

Q3. How is the lifetime of an object determined? What happens to an object when it dies?

Ans The lifetime of an object in a programming language is determined by its memory allocation and deallocation mechanism. In language like Python, Java and C#, objects are managed by a garbage collector, while in languages like C and c++, memory management is done manually by the Programmer.

DATE: / /
PAGE NO.: 43

- In Python specifically :-

(i) Memory Allocation : When an object is created, memory is allocated for it. This memory allocation can happen implicitly when an object is instaiated using a class constructor, or explicitly when memory is allocated dynamically.

(ii) Reference Counting :- Python uses a reference counting mechanism to keep track of how many references (variables, data structure, etc.) point to an object. When the reference count drops to zero, it means that there are no more references to the object, and it becomes eligible for garbage collection.

(iii) Garbage Collection : Python has a garbage collector that periodically checks for objects with a reference count of zero. When such objects are found, the memory they occupy is reclaimed and made available for future use.

When an objects "dies" in Python, it means that its lifetime has ended, and it is no longer accessible or usable. This typically happens when there are no more references to the object, indicating that it is no longer needed by the program. Once the object's memory is reclaimed by the garbage collector, the object is essentially deleted from memory, and its resources are freed up for other use.

- The process of an object "dying" involves the following steps:

(i) Reference Count Drops to Zero:
(ii) Garbage Collection
(iii) Memory Reclamation
(iv) object deletion.

Part - C (11*3 = 33)

1. Write a Python program to find all words starting with 'a' or 'e' in a given string.

Ans #
```
def find_words(text):
    words = [word.lower() for word in text.split()]
    return [word for word in words if word[0] in ("a", "e")]

text = "He is ankit and he earns ten thousand everyday."
matching_words = find_words(text)
print("words starting with 'a' or 'e': ", matching_words)
```

O/P:- words starting with "a" or 'e' : ['ankit', 'and', 'earns', 'everyday']

OR with User Input:-

#
```
def find_words(text):
    words = text.lower().split()
    return [word for word in words if word[0] in ("a", "e")]

text = input("Enter a string: ")

matching_words = find_words(text)
If matching_words:
    print("words starting with 'a' or 'e':", matching_words)
else:
    print("No words found starting with 'a' or 'e'.")
```

O/P: Enter a string: i am ankit
word starting with 'a' or 'e' : ['am', 'ankit']

DATE: / /
PAGE NO.: 45

Q2 Write a Python Program to ~~find all words starting~~ Search for literal strings within a string.
Sample text: 'The quick brown fox jumps over the lazy dog.'
Searched words: 'fox', 'dog', 'horse'

```
text = "The quick brown fox jumps over the lazy dog."
searched_words = ["fox", "dog", "horse"]

for word in searched_words:
    if word in text:
        print(f"Found '{word}' in the text.")
    else:
        print(f"'{word}' not found in the text.")
```

o/p: Found 'fox' in the text.
Found 'dog' in the text.
'horse' not found in the text.

OR

```
def search_str(text, searched_words):
    found_words = []
    for word in searched_words:
        if word in text:
            found_words.append(word)
    return found_words

sample_text = 'The quick brown fox jumps over the lazy dog.'
searched_words = ['fox', 'dog', 'horse']
found_words = search_str(sample_text, searched_words)
print("Found words:", found_words)
```

output:

Found words: ['fox', 'dog']

Q3. Complete the code given below and then perform the following tasks on the Coordinate class.

(a) instantiate two different objects P1 and P2.

(b) Display the coordinates of P1 and P2.

c. Add an --eq-- method that returns True if Coordinate P1 and P2 refer to the same point in a plane.

```
class Coordinate(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def getX(self):
        return self.x
    def getY(self):
        return self.y
```

Ans

```
class Coordinate(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def getX(self):
        return self.x
    def getY(self):
        return self.y

    def __eq__(self, other):
        if isinstance(other, Coordinate):
            return self.x == other.x and self.y == other.y
        return False

P1 = Coordinate(3,4)
P2 = Coordinate(3,4)
```

```
print(f"Coordinates of P1 : ({P1.getX()}, {P1.getY()})")
print(f"Coordinates of P2 : ({P2.getX()}, {P2.getY()})")

if P1 == P2:
    print("P1 and P2 refer to the same point.")
else:
    print("P1 and P2 refer to different points.")
```

True,

False

O/P:

Coordinate of P1 : (3,4)

Coordinate of P2 : (3,4)

True, P1 and P2 refer to the same point.